

SiFlex 4.0

Especificação Técnica e Arquitetural

Versão 0.1 — Documento inicial de fundação

Status: Documento vivo de projeto; será atualizado conforme novas decisões forem aprovadas.

Objetivo: construir uma nova versão do SiFlex em paralelo à versão 3.0, sem alterar o sistema atual, criando uma arquitetura mais robusta e preparada para uma futura migração controlada dos dados e funcionalidades.

1. Objetivo do projeto

- O SiFlex 4.0 será uma reconstrução arquitetural do sistema, desenvolvida paralelamente ao SiFlex 3.0. A versão atual continuará operando normalmente durante todo o desenvolvimento da V4.
- A V4 não será uma simples cópia do código existente. O objetivo é preservar regras de negócio e compatibilidade funcional necessárias, enquanto se corrigem limitações históricas e se estabelece uma base mais organizada, segura, testável e sustentável.

2. Princípios da V4

- Desenvolvimento paralelo: a V3 permanece intacta e em produção.
- Arquitetura modular: infraestrutura e funcionalidades de negócio serão claramente separadas.
- Segurança desde a fundação: autenticação, autorização, sessões, validação e auditoria serão projetadas desde o início.
- Compatibilidade de dados: toda nova estrutura deverá permitir mapeamento futuro dos dados da V3.
- Versionamento: código, banco, documentação e migrações deverão ser versionáveis.
- Evitar dívida técnica deliberada: decisões arquiteturais importantes serão documentadas.
- Reutilização do conhecimento da V3, sem reproduzir automaticamente suas limitações históricas.

3. Relação entre V3 e V4

- A V3 será utilizada como referência funcional para compreender tabelas, procedures, telas, regras de negócio, permissões e relacionamentos.
- A V4 possuirá seu próprio banco e sua própria arquitetura. O banco V3 não será tratado como estrutura obrigatória para a aplicação V4.
- A futura migração seguirá o conceito Extract → Transform → Validate → Load, permitindo transformar os dados antigos para as novas estruturas.

4. Arquitetura geral

- Arquitetura conceitual: Frontend → API/Backend → Core/Serviços → Banco de dados e armazenamento.
- O Core será responsável por capacidades transversais, como autenticação, autorização, conexão com banco, logs, configurações, tratamento de erros e gerenciamento de arquivos.
- Os módulos de negócio utilizarão o Core em vez de duplicar essas funcionalidades.

5. Estrutura inicial de diretórios

```
siflex4/
├── frontend/
│   ├── assets/
│   ├── css/
│   ├── js/
│   └── index.html
├── backend/
│   ├── api/
│   ├── core/
│   ├── modules/
│   ├── config/
│   └── database/
```

```
| └─ schema/  
| └─ procedures/  
| └─ seeds/  
| └─ migrations/  
└─ modules/  
└─ storage/  
└─ migration/  
    └─ v3/  
└─ tests/  
└─ docs/  
└─ config/
```

6. Frontend

- O frontend será responsável pela interface, navegação, apresentação, validações de experiência e comunicação com a API.
- A aplicação deverá possuir uma interface base capaz de carregar e controlar módulos sem que cada funcionalidade precise reinventar a estrutura principal.
- O frontend não será responsável por regras de segurança; toda autorização real deverá ser validada no backend.

7. Backend

- O backend será a camada responsável pela API, autenticação, autorização, validação, infraestrutura, acesso ao banco, logs e integrações externas.
- As funcionalidades serão organizadas por módulos, mantendo o Core independente das regras específicas de cada área do ERP.

8. API

- A comunicação frontend/backend será feita por uma API HTTP, preferencialmente utilizando JSON como formato padrão de entrada e saída.
- A API deverá possuir respostas padronizadas para sucesso, validação, autenticação, autorização e erros internos.
- A autenticação e as permissões serão verificadas no servidor.

9. Banco de dados V4

- O banco V4 será independente do banco V3, ainda que sua modelagem seja baseada no conhecimento adquirido no sistema atual.
- As alterações do banco deverão ser versionadas e organizadas em arquivos de schema, procedures, seeds e migrations.
- Estruturas históricas da V3 poderão ser remodeladas quando houver benefício claro, desde que seja possível definir posteriormente um mapeamento para migração.

10. Autenticação

- A autenticação será um dos primeiros módulos funcionais da V4.
- Login e validação de credenciais.
- Gerenciamento de sessão.
- Logout.
- Expiração e controle de sessão.

- Alteração de senha.
- Recuperação/redefinição de senha.
- Autenticação será separada conceitualmente de autorização.

11. Usuários

- O gerenciamento de usuários será construído sobre o sistema de autenticação e permitirá criação, edição, ativação, bloqueio, redefinição de senha e gerenciamento das associações de acesso.
- A estrutura V4 deverá permitir que os registros de usuários da V3 sejam posteriormente transformados por um script de migração.

12. Perfis e permissões

- A V4 separará Usuário, Perfil, Permissão e Módulo como conceitos distintos.
- Modelo conceitual: Usuário → Perfil → Permissões → Módulos.
- Permissões poderão ser granulares, contemplando visualizar, criar, editar, excluir, exportar, imprimir e aprovar, quando aplicável.

13. Sistema de módulos

- Cada funcionalidade principal será tratada como um módulo identificável.
- O módulo deverá possuir metadados como identificação, nome, descrição, ícone, rota, status e ordem de apresentação, conforme a necessidade.
- O sistema de módulos permitirá que menu e acesso sejam construídos dinamicamente.

14. Menu

- O menu deverá ser derivado dos módulos disponíveis e das permissões do usuário.
- O frontend não será a fonte definitiva das permissões; o backend deverá impedir acesso direto a operações não autorizadas.

15. Configurações

- A V4 deverá distinguir, quando necessário, configurações globais, por empresa, por usuário e por módulo.
- A necessidade de arquivos JSON será avaliada caso a caso; configurações importantes deverão ter estratégia de persistência claramente definida.

16. Arquivos e armazenamento

- O sistema deverá possuir uma camada de armazenamento separada da lógica dos módulos.
- Operações de upload, leitura, renomeação e exclusão deverão passar por validação e autorização no backend.
- A estrutura deverá facilitar futura separação entre arquivos públicos, privados e documentos de usuários/empresas.

17. Logs e auditoria

- Logs e auditoria serão implementados cedo no projeto.
- Quando aplicável, a auditoria deverá registrar usuário, data/hora, operação, módulo, registro afetado e alterações relevantes.

18. Tratamento de erros

- A API deverá utilizar respostas padronizadas e não expor detalhes internos, SQL, caminhos de filesystem ou credenciais.
- Erros deverão ser registrados internamente com informações suficientes para diagnóstico, enquanto o frontend receberá mensagens seguras e úteis.

19. Segurança

- Senhas nunca deverão ser armazenadas em texto puro.
- Credenciais e segredos de serviços externos não deverão ficar expostos no frontend.
- Autorização deverá ser aplicada no backend.
- Entradas deverão ser validadas de acordo com o contexto.
- Operações com banco deverão utilizar mecanismos seguros contra injeção.
- Sessões deverão possuir expiração e invalidação controladas.
- Operações de arquivos deverão validar caminhos, tipos e permissões.
- Informações sensíveis não deverão ser incluídas em logs desnecessariamente.

20. Testes

- A V4 deverá possuir estratégia de testes desde o desenvolvimento inicial.
- Testes de API e autenticação.
- Testes de autorização.
- Testes das regras de negócio.
- Testes de banco.
- Testes de migração.
- Testes de regressão para funcionalidades portadas da V3.

21. Migração V3 → V4

- A migração será tratada como projeto próprio e desenvolvida paralelamente à evolução da V4.
- Estrutura inicial: migration/v3/, mappings/, scripts/ e documentação.
- Fluxo previsto: extração dos dados V3 → transformação → validação → carga na V4 → validação pós-carga.
- A migração não deverá alterar o banco de produção da V3 durante os testes. Devem existir mecanismos para validar contagens, relacionamentos, integridade e registros críticos.

22. Versionamento

- Código, banco, documentação e scripts de migração deverão ser versionados.
- Alterações arquiteturais importantes deverão ser registradas neste documento ou em documentos técnicos vinculados.

23. Padrões de desenvolvimento

- Antes de implementar um novo módulo, deverão estar disponíveis as capacidades de infraestrutura necessárias.
- Módulos não deverão duplicar funcionalidades existentes no Core.

- Regras de negócio importantes deverão ter testes ou documentação suficiente para evitar regressões.
- A V3 será utilizada como referência funcional, e não como modelo obrigatório de implementação.

24. Roadmap inicial

- 01 — Fundação e arquitetura.
- 02 — Banco/Core.
- 03 — API.
- 04 — Autenticação.
- 05 — Usuários.
- 06 — Perfis e permissões.
- 07 — Gerenciamento de módulos.
- 08 — Menu dinâmico.
- 09 — Logs e auditoria.
- 10 — Configurações.
- 11 — Empresas.
- 12 — Setores, cargos e unidades.
- 13 — Funcionários.
- 14 — Produtos.
- 15 — Estoque.
- 16 — Compras.
- 17 — RH.
- 18 — Financeiro.
- 19 — Operacional.
- 20 — Fiscal.
- 21 — Integrações.
- 22 — Relatórios.
- 23 — Migração V3 → V4.
- 24 — Testes de migração.
- 25 — Homologação.
- 26 — Transição/cutover da V3 para a V4.

25. Decisões arquiteturais e pendências

- Escolha final da tecnologia/estrutura do frontend.
- Organização definitiva do backend e padrão de rotas da API.
- Estratégia exata de sessão e armazenamento do estado de autenticação.
- Modelo definitivo das tabelas de usuários, perfis, permissões e módulos.
- Estratégia de versionamento e execução das migrations do banco.
- Estratégia de armazenamento de arquivos.
- Política definitiva de logs e auditoria.
- Estratégia de testes automatizados.
- Procedimento formal de migração e validação V3 → V4.